MCP + RAG для 1С

Локальный поиск по BSL-коду и метаданным:
гибрид BM25 + cosine через RRF и честная оценка качества

Sergey Matyukhin

github.com/Smaturus/1c-mcp-rag-search

MCP Python SDK · Chroma · multilingual-e5-small · BM25 · RRF · flashrank

Задача и что получилось

Проблема

- Выгрузка 1С — тысячи .bsl-модулей и отчёт на десятки мегабайт
- Обычный grep не отвечает на смысловые вопросы
- Готовые поисковые решения требуют развёртывания и внешних API
- LLM-агенту нужен инструмент с предсказуемым качеством

Решение

- Свой MCP-сервер (stdio): 8 инструментов, включая маршрутизацию запросов
- Полностью локальный контур: e5-small + Chroma, работает на CPU
- Гибридный поиск: cosine + BM25 через RRF; метаданные — из XML-выгрузки
- Измеримость: gold-set, file-level метрики, разброс по прогонам

Проект в цифрах

179 586
чанков в индексе

0.339
nDCG@10 у vector

68/68
кейсов маршрутизации

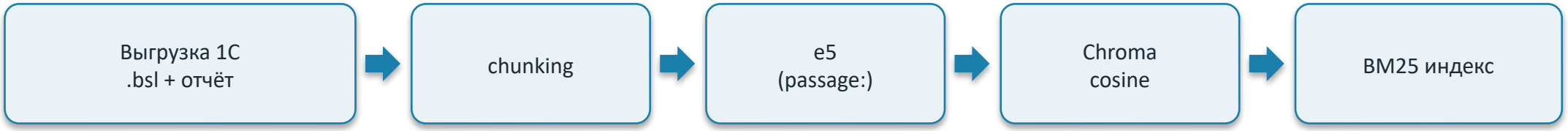
99,7 %
BM25 Hit@3 метаданных

Стек

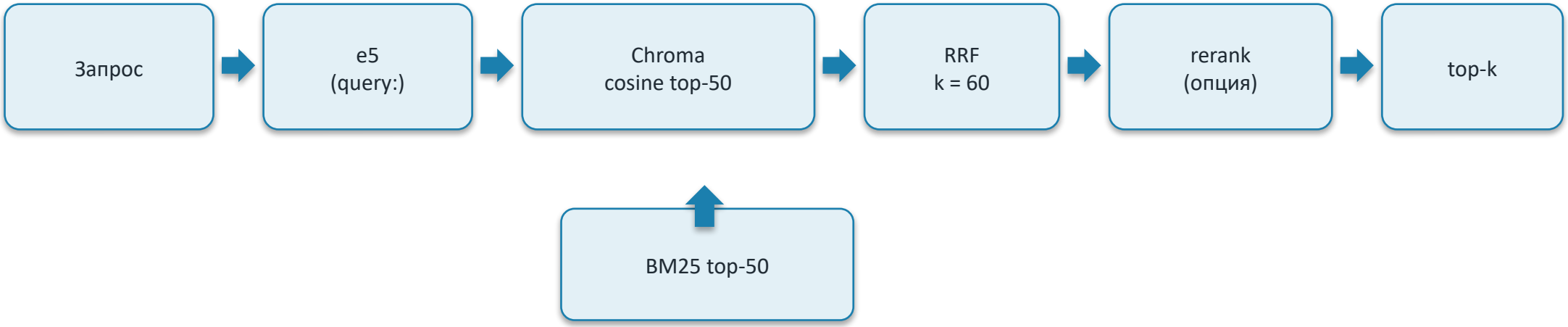
- Эмбеддинги: intfloat/multilingual-e5-small (384d, префиксы query: / passage:)
- Хранение: Chroma 1.5.9, HNSW cosine; лексика: собственный инвертированный индекс
- Гибрид: Reciprocal Rank Fusion (k = 60); reranker: flashrank + ms-marco-MultiBERT-L-12
- Интерфейс: MCP stdio (Cursor / ZCode), 4 инструмента; eval: 70 запросов, 4 конфигурации

Архитектура: два контура

Индексация (offline)



Поиск (online, MCP stdio)



Сверху — офлайн-подготовка индексов; ниже — онлайн-путь запроса. RRF объединяет два независимых ранжирования, reranker подключается опционально. Все данные и вычисления локальные: внешних API нет.

Индексы: векторы и лексика

Chroma (векторный)

- 179 586 чанков: code 154 560 (процедуры), metadata 25 026 (XML)
- e5-small: 384 измерения, лимит 512 токенов на чанк
- Проверка: 32,1 % чанков были длиннее лимита — исправлено нарезкой по процедурам (ADR 0007)
- Префиксы e5 сохранены: passage: для документов, query: для запросов
- HNSW cosine; sync_threshold = 1e6 — защита от бага chroma#7090
- Размер базы 2.3 ГБ; старт пересобирает HNSW из WAL (~60–70 с)

BM25 (лексический)

- Инвертированный индекс: 219 968 термов, 6,6 млн постингов
- Сборка 22 с, 58 МБ на диске, читает документы прямо из sqlite
- Токенизация под 1С: кириллица, цифры, коды через дефис (НФ-001)
- Скоринг только по постингам запроса — без полного перебора корпуса
- Используется как второй источник ранжирования в RRF

MCP-сервер: инструменты для агента

Инструмент	Что делает
search_1c_metadata(query, limit)	grep-подобный BM25: точные имена, проводки, движения
search_knowledge_base(query, limit)	vector: «как сделать», «почему не работает», инструкции
search_hybrid(query, limit)	RRF: лучший для exact-имён; на новом корпусе vector впереди
route_query(query)	объясняет выбор: маршрут, причина, уверенность
search / index_stats / evaluate_retrieval	поиск, состав индекса и метрики по запросу

Подключение

- stdio-транспорт: сервер поднимается клиентом (Cursor mcp.json, ZCode mcp.servers)
- Маршрутизация — код с тестами: 68 кейсов, 100 %; blind 20/20 после правок
- Пример: route_query("проводки документа РеализацияТоваровУслуг") → search_1c_metadata

Гибридный поиск: Reciprocal Rank Fusion

$$score(d) = \Sigma\ 1 / (k + rank_i(d)),\quad k = 60$$

Почему RRF

- Cosine и BM25 дают скоры в несравнимых шкалах
- RRF использует только ранги — калибровка не нужна
- Точные имена ловит BM25, смысл — cosine, RRF складывает оба
- k = 60 сглаживает вклад первых позиций

Что дало (nDCG@10)

- Прежний корпус: hybrid 0.209 (+19 % к vector)
- Новый корпус: vector 0.339, hybrid 0.301
- Причина: процедурные чанки ослабили BM25 (ADR 0009)
- Латентность: 50–120 мс/запрос

Пример. Запрос «ПередЗаписью»: BM25 поднимает модули с точным именем обработчика, cosine — смысловые совпадения. RRF складывает оба ранжирования без подбора весов.

Reranker: отрицательный результат, оставленный в отчёте

Как реализован

- flashrank 0.2.10, ONNX на CPU, без PyTorch
- ms-marco-MultiBERT-L-12, max_length = 128
- Вход: 20 кандидатов RRF; пассаж = путь файла + код
- Ленивая загрузка модели, кэш на диске

Результат

- nDCG@10: 0.301 → 0.117 (−61 %)
- Латентность: ~2,4 с/запрос (×20 к vector)
- Гипотеза: модель обучена на веб-пассажах, а BSL — длинный повторяющийся код
- Решение: режим оставлен опцией; дефолт пересматривается (ADR 0002, 0009)
- Пайплайн не «крутился» под результат: проигравшая конфигурация осталась в таблице

Как измеряли качество

Данные и метрики

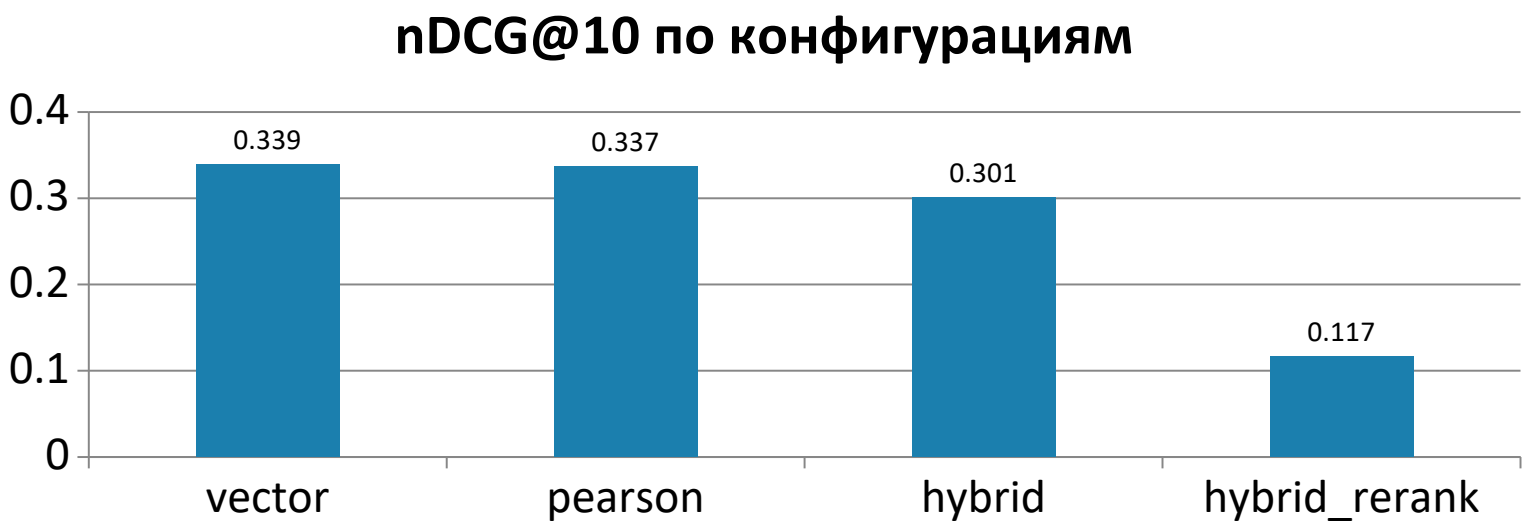
- gold-set v2: 70 запросов (30 курируемых + 40 по объектам конфигурации)
- Три типа: exact (точное имя), semantic (смысл), trap (ловушка)
- Релевантность — на уровне файла-источника, а не отдельного чанка
- Метрики: Hit@1, MRR@10, nDCG@10, Recall@10
- Каждый прогон — новый процесс: HNSW пересобирается; 4 прогона, mean ± std

Почему так

- Чанковые эталоны давали ложно-нулевые метрики: правильный файл «не засчитывался»
- Файловая релевантность устойчива к смене chunk_size
- Recall@k нормирован на min(|relevant|, k)
- Разброс ±0.023 у vector — вывод воспроизводим, а не случайная удача

Результаты на gold-set v2 (70 запросов, k = 10)

Конфигурация	Hit@1	MRR@10	nDCG@10	Recall@10	мс/запрос
vector	0.150 ± 0.007	0.268 ± 0.019	0.339 ± 0.023	0.523 ± 0.030	~60
pearson	0.179 ± 0.021	0.270 ± 0.024	0.337 ± 0.030	0.511 ± 0.048	~90
hybrid (RRF)	0.129 ± 0.010	0.240 ± 0.013	0.301 ± 0.017	0.482 ± 0.034	~70
hybrid_rerank	0.086	0.102	0.117	0.157	~2400



Источник: data/eval/code_proc/report_pipeline.md

vector — лидер на новом корпусе.
hybrid лучший для exact-имён
(0.408 против 0.250 по nDCG@10).

Reranker ухудшает качество.

Решения и ограничения

Decision Log (ADR)

- 0001: RRF вместо взвешенного Pearson — выше метрики, без калибровки весов
- 0002: reranker оставлен опцией — ухудшает качество на BSL
- 0003: релевантность на уровне файла — устойчивые и воспроизводимые метрики
- 0006: маршрутизация запросов — код с тестами вместо текста промпта
- 0007: нарезка по процедурам — устранён лимит 512 токенов
- 0008: метаданные из XML — BM25 Hit@3 93,0 → 99,7 %
- 0009: смена корпуса — лидер hybrid → vector, зафиксировано

Ограничения

- HNSW не персистится на диск: старт процесса ~60–70 с (защита от chroma#7090)
- Gold-set частично синтетический, абсолютные метрики скромные
- Один писатель индекса; параллельная индексация не поддерживается
- Нет инкрементальной индексации: полная пересборка — часы
- Reranker для кода — следующий контур (модель на коде или fine-tune)

Итоги и развитие

Что сделано

- MCP-сервер с 8 инструментами и локальным индексом на 179 тыс. чанков
- Гибридный retrieval: глобальный BM25 + RRF + опциональный cross-encoder
- Маршрутизация: контракт с тестами, 68 кейсов + blind 20/20
- Eval-контур: gold-set v2 (70 запросов), file-level метрики, mean ± std
- Инженерная обвязка: 72 теста, ruff, mypy, complexity-гейт, CI, ADR, CHANGELOG

Развитие

- Ablation нарезок и веса RRF; затем смена дефолта на vector
- Инкрементальная индексация и структурные инструменты (get_metadata, find_callers)
- Код-ориентированные эмбеддинги; reranker на коде; ручная разметка 100+ запросов

github.com/Smaturus/1c-mcp-rag-search · smaturus@gmail.com